# TECNOLOGIE E ARCHITETTURE PER LA GESTIONE DEI DATI
# A.A. 2024/2025

# METODI DI INGEGNERIZZAZIONE DELLE PRESTAZIONI

Capitolo 4



P. ATZENI, L. BELLOMARINI, S. IANNUCCI

# INTRODUZIONE

Nelle precedenti lezioni abbiamo imparato i concetti di base dell'analisi operazionale. Dato un sistema:

- Sappiamo scegliere la tipologia di QN per modellarlo
- Sappiamo come misurare e/o ricavare dati relativi a:
  - Utilizzazione dei suoi componenti
  - Service demand
  - Tempo di risposta
  - Throughput
  - …

Vediamo ora come usare questa conoscenza per ingegnerizzare le prestazioni di un sistema in tutte le fasi del suo ciclo di vita.

# INGEGNERIZZAZIONE DELLE PRESTAZIONI

Prestazioni solitamente tenute in considerazione soltanto nelle fasi finali dello sviluppo di un software o di un sistema

- Conseguenze: (1) è molto probabile che il sistema non sia sufficientemente veloce o affidabile, oppure (2) che il sistema sia costato e/o stia costando molti più soldi di quanto non sia necessario spendere

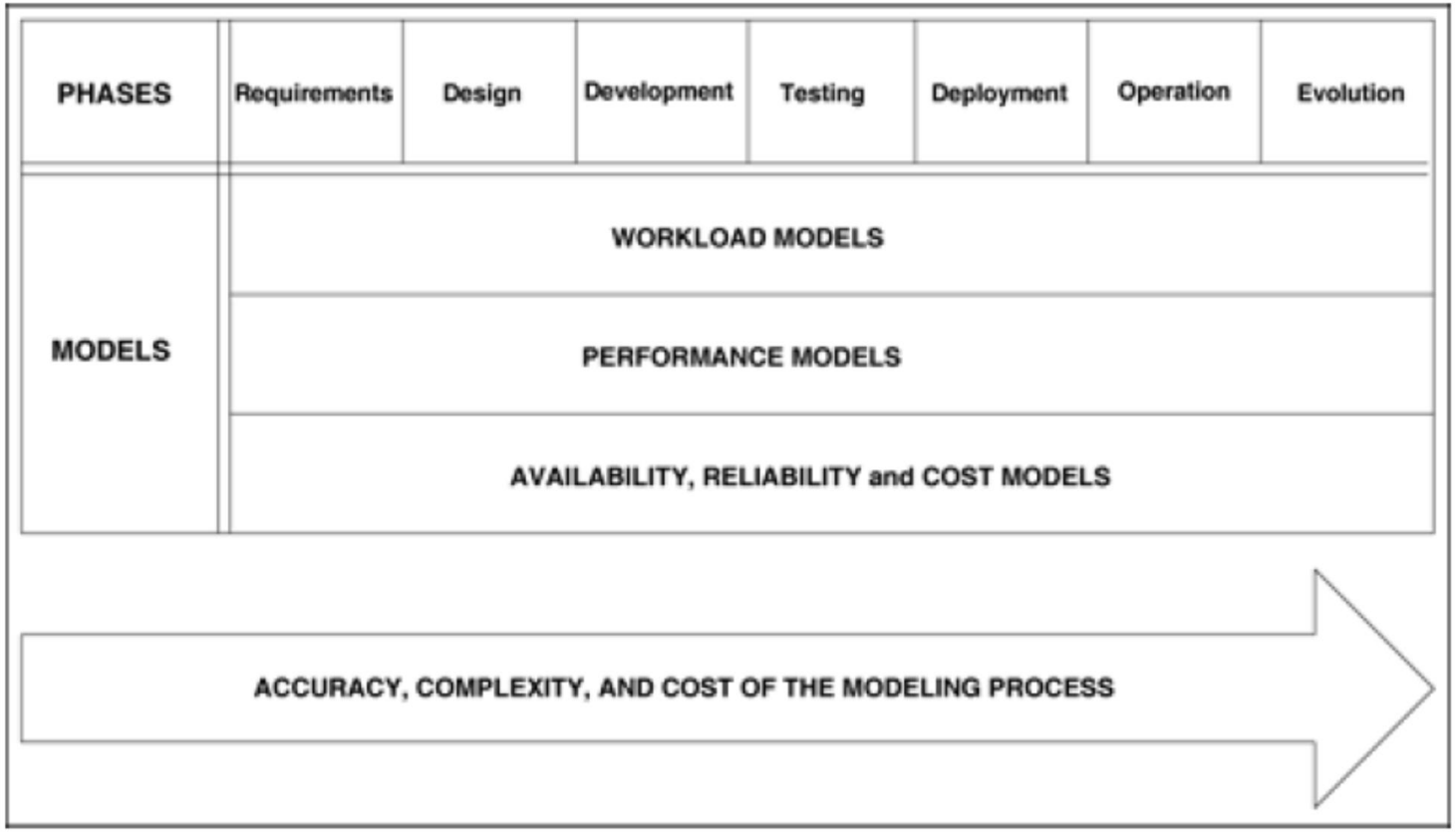
L'ingegneria delle prestazioni si occupa, in ogni fase del ciclo di vita di un sistema, di:

- Sviluppare strategie pratiche che aiutino nella predizione delle prestazioni che un sistema può raggiungere
- Fornire raccomandazioni per raggiungere il livello di prestazioni ottimale

Processo basato sulle seguenti attività:

- Misurazione delle metriche di sistema e comprensione del workload al quale è soggetto
- Sviluppo e validazione di un modello di workload che catturi le caratteristiche del workload reale
- Sviluppo e validazione di un modello analitico e/o simulativo che possa predire accuratamente le prestazioni del sistema
- Uso dei modelli per predire ed ottimizzare le prestazioni del sistema

# FASI DEL PROCESSO DI MODELLAMENTO



# ESEMPIO MOTIVAZIONALE: REQUISITI FUNZIONALI

Un'azienda che produce parti di ricambio per auto vuole creare un nuovo call center con i seguenti obiettivi:

- Incoraggiare un migliore rapporto con i clienti allo scopo di fidelizzarli
- Migliorare l'efficienza dei servizi offerti
- Identificare ed esplorare nuove possibilità di vendita.

Il team IT ha recepito questa volontà ed ha proposto la creazione di un nuovo Customer Relationship Management system (CRM), integrato con il centralino telefonico.

Gli operatori dovranno rispondere a domande del tipo:

- Stato di un ordine
- Luogo dove si trova una spedizione
- Stato di risoluzione di un problema

Ed avranno a disposizione:

- Cronologia di tutti i contatti avuti con i clienti
- Visualizzazione dettagliata delle informazioni di un cliente
- Informazioni riguardo la risoluzione di problemi passati
- Altre funzionalità di aiuto.

# ESEMPIO MOTIVAZIONALE: REQUISITI NON FUNZIONALI

L'azienda si rende conto che i clienti non vogliono restare in attesa per lungo tempo al telefono. Quindi solleva le seguenti preoccupazioni:

- Il sistema sarà in grado di risponde in meno di un secondo alle query degli operatori?
- Quale sarà l'impatto del raddoppiare il numero di operatori l'anno successivo?
- Le prestazioni del CRM rimarranno soddisfacenti dopo che questo verrà integrato con il sistema di gestione dell'inventario già esistente?
- Il sistema sarà in grado di gestire fino a 1000 chiamate nell'ora di punta mantenendo un tempo di risposta inferiore al secondo?
- Che impatto avrà un eventuale fallimento del server DB rispetto all'obiettivo di una disponibilità 24x7?
- Che impatto avrà offrire ai clienti un servizio basato su web, in aggiunta a quello telefonico?

# FASE DI ANALISI DEI REQUISITI

In questa fase possiamo lavorare su:

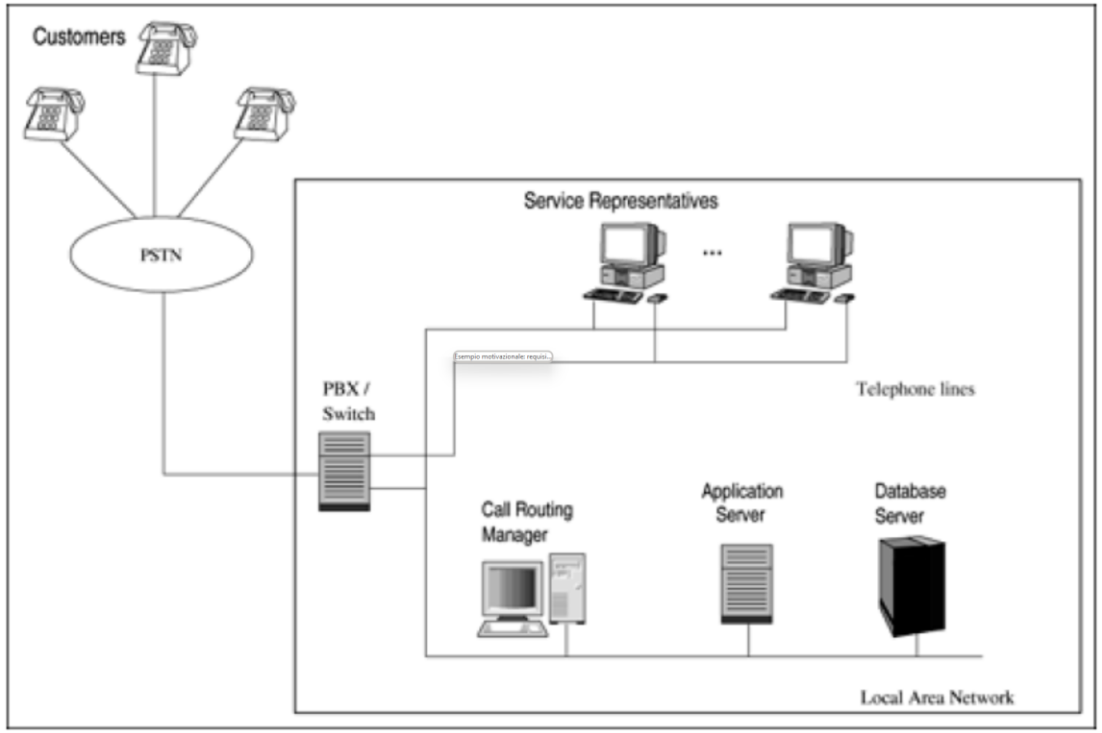
- Architettura del sistema
- Definizione del workload

Ad esempio, per questo progetto è ipotizzabile un'architettura a due livelli: application server, database server.

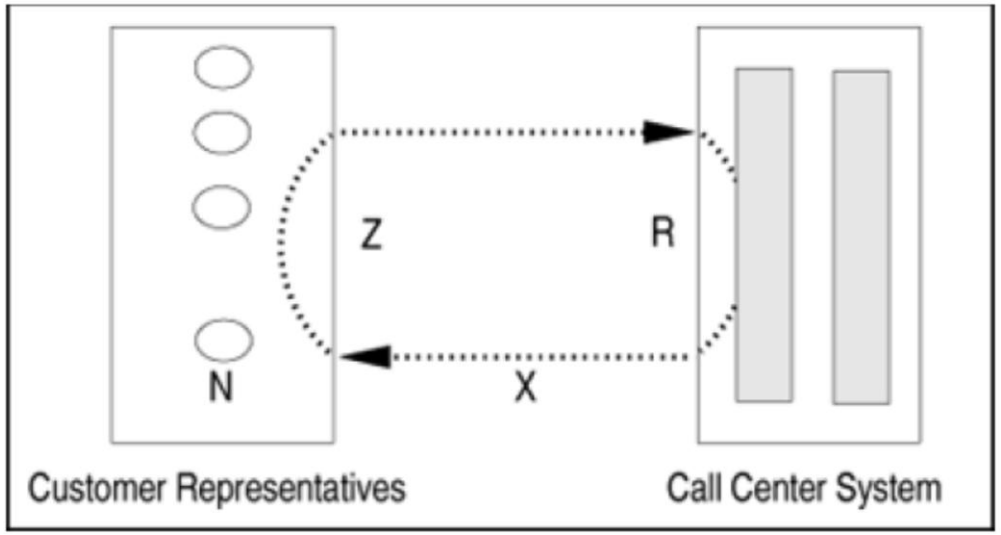
Il workload non è definito in modo puntuale nei requisiti. Però, possiamo già fare alcune considerazioni:

- Avremo almeno tre tipi di workload, ognuno con le proprie unità di misura: cliente→operatore; operatore → application server; application server → database server
- Tipi di workload più dettagliati (e.g., operazioni su disco, tempo di CPU per funzione, etc) saranno disponibili solo successivamente.

Non avendo altri dettagli, il modello delle prestazioni è per noi ancora una black box



# FASE DI PROGETTAZIONE




Anche se il sistema è ancora una black box, ci sono già calcoli molto interessanti che possiamo fare.

- Ad esempio, il management ci dice che ci saranno 200 operatori, e che nei periodi di picco sarà presente l'80% di questi per rispondere alle chiamate.
- Il processo può essere rappresentato con un modello interattivo:
  1) Il cliente fa una domanda all'operatore, il quale la traduce in una funzione da eseguire sul sistema
  2) La richiesta viene inviata al sistema, che la elabora ed invia una risposta all'operatore
  3) L'operatore dialoga con il cliente ed eventualmente invia altre richieste al sistema
- Stimando un think time $Z = 30s$, siamo già in grado di stabilire quale debba essere il throughput minimo del sistema:
  - Legge del tempo di risposta interattivo: $R = \frac{N}{X_0} - Z \leq 1$

$$\Rightarrow X_0 \geq \frac{N}{Z+1} = \frac{200 \times 0.8}{30+1} = 5.16\, funzioni/sec$$

# FASE DI SVILUPPO

Nuovi dettagli emergono.

- Ogni funzione invocata dall'operatore esegue in media 2.2 query sul database.

Questo vuol dire che il throughput minimo del DB dovrà essere pari a:

- Forced Flow Law: $X_{DB} = V_{DB} \times X_0 = 2.2 \times 5.16 = 11.35\ tps$

# FASE DI PRODUZIONE

Vogliamo garantire le prestazioni del sistema.

- Cominciamo con il tenere traccia dei tempi di risposta
$$R_{call} = R_{appl} + R_{LAN} + R_{DB}$$
- Come requisito di progetto abbiamo che: $R_{call} < 1$
  Sfortunatamente, osserviamo che durante le ore di picco, il tempo di risposta sale oltre 1 secondo.
  Se volessimo fare un upgrade, qual è il componente che dovremmo cambiare?
- Per semplicità supponiamo di avere una sola classe di query, e che durante il periodo di picco
$$\lambda = 57600\ q/h$$
  - Osserviamo che, in media, ogni query usa 50ms ($D_{CPU} = 50ms$) di CPU e fa 4 operazioni di I/O su Disco1 e 2 su Disco2. Ogni operazione di I/O impiega 8ms.
- Supponendo un sistema in equilibrio, $X_0 = \lambda_0 = 16qps$
  - $D_{Disco1} = V_1 \times S_{Disco1} = 4 \times 0.008 = 0.032s$
  - $D_{Disco2} = V_2 \times S_{Disco2} = 2 \times 0.008 = 0.016s$

# FASE DI PRODUZIONE (2)

Avendo i service demand ed il throughput, con la *service demand law* possiamo calcolare l'utilizzazione:

- $U_{CPU} = D_{CPU} \times X_0 = 0.05 \times 16 = 0.8$
- $U_{Disco1} = D_{Disco1} \times X_0 = 0.032 \times 16 = 0.512$
- $U_{Disco2} = D_{Disco2} \times X_0 = 0.016 \times 16 = 0.256$

Usando l'equazione per i tempi di residenza (per il momento prendetela per buona) **per reti aperte,** possiamo calcolare il tempo di residenza in ogni centro:

- $R_{CPU} = \frac{D_{CPU}}{1-U_{CPU}} = \frac{0.05}{1-0.8} = 0.25s$
- $R_{Disco1} = \frac{D_{Disco1}}{1-U_{Disco1}} = \frac{0.032}{1-0.512} = 0.066s$
- $R_{Disco2} = \frac{D_{Disco2}}{1-U_{Disco2}} = \frac{0.016}{1-0.256} = 0.022s$

Otteniamo quindi che: $R_{DB} = 0.25 + 0.066 + 0.022 = 0.338s$

Ricordiamo che $R_{call} = R_{appl} + R_{LAN} + R_{DB} < 1.$

# FASE DI EVOLUZIONE

Si vuole estendere il sistema introducendo un'applicazione web.

- Questo comporta carico addizionale rispetto alle telefonate ricevute dal call center, pari a 1 query al secondo.

Nello specifico, si stimano i parametri riportati in tabella.

E' possibile aggiungere questo workload senza fare un upgrade del sistema?

| | Local Queries | Web Queries |
|---|---|---|
| Arrival Rate (tps) | 16 | 1 |
| Service Demands (sec) | | |
| CPU | 0.05 | 0.15 |
| Disco1 | 0.032 | 0.2 |
| Disco2 | 0.016 | 0.1 |